1.

(a) Valid. Deterministic and maps into $\{0, ..., N-1\}$ (all keys collide, but it is a legal hash function).
(b) Invalid. Not deterministic: the same key may map to a different slot at search time.
(c) Valid. This is the division method.
(d) Invalid. Range is $\{0, ..., N-3\}$: slots $N-2$ and $N-1$ are never used.
(e) Valid as a function (deterministic, range $\{0, ..., N-1\}$), but unusable: every operation costs $\Theta(N^2)$ just to evaluate h.

2. Let $x = x_{r-1}...x_1x_0$ with $x_i \in \{0, ..., 2^p - 1\}$:

$$k(x) = \sum_{i=0}^{r-1} x_i \cdot (2^p)^i. \tag{1}$$

Since $2^p \equiv 1 \pmod{2^p - 1}$, every power satisfies $(2^p)^i \equiv 1 \pmod{m}$, so

$$h(x) = \left(\sum_{i=0}^{r-1} x_i \right) \bmod m, \tag{2}$$

which depends only on the sum of the characters, not their positions. Hence permuting the characters of y gives $h(x) = h(y)$. □

3.

(a) False. Open addressing: $\frac{1}{1-\alpha}$ probes; chaining: $1 + \alpha$ probes. Since $\frac{1}{1-\alpha} > 1 + \alpha$ for all $0 < \alpha < 1$, chaining is faster.
(b) False. Insertion costs $\frac{1}{1-\alpha}$ probes vs. $1 + \alpha$ in chaining; deletion in open addressing needs tombstones or shifting, so it is not faster either.
(c) True. No pointers per slot/element; for the same storage a larger table (lower load factor) is possible.
(d) False. Open addressing: the initial probe finds an occupied slot with probability exactly α. Chaining: a new key lands in a non-empty bucket with probability $1 - \left(1 - \frac{1}{m}\right)^n \approx 1 - e^{-\alpha} = \alpha - \frac{\alpha^2}{2} + \cdots < \alpha$ (and under the literal definition "no empty slot found", chaining never has one at all). Not equal.

4. Expected empty slots:

$$m\left(1 - \frac{1}{m}\right)^n \approx me^{-\frac{n}{m}}. \tag{3}$$

(a) $m = \sqrt{n}$: $\sqrt{n}e^{-\sqrt{n}} \to 0$ — **(ii) zero**.
(b) $m = cn$: $cne^{-\frac{1}{c}} = \Theta(n) \to \infty$ — **(iii) infinity**.
(c) $m = n^a$, $a > 1$: $\frac{n}{m} \to 0$, so $n^a(1 - o(1)) \to \infty$ — **(iii) infinity**.

5. Linear probing, $h(k) = k \bmod m$, initial $m = 4$, double when load factor would exceed 0.75.

Insert	**Trace**	**Placement**	**Resize?**
12	$12 \bmod 4 = 0$	slot 0	no ($\frac{1}{4}$)
43	$43 \bmod 4 = 3$	slot 3	no ($\frac{2}{4}$)
13	$13 \bmod 4 = 1$	slot 1	no ($\frac{3}{4} = 0.75$)
88	$88\text{mod}4 = 0 \to 0, 1$ busy $\to 2$	slot 2	$\frac{4}{4} = 1 > 0.75$: double to $m = 8$
23	$23 \bmod 8 = 7$	slot 7	no ($\frac{5}{8}$)

26	$26 \bmod 8 = 2$	slot 2	no ($\frac{6}{8} = 0.75$)
11	$11 \bmod 8 = 3 \to 3, 4, 5$ busy $\to 6$	slot 6	$\frac{7}{8} > 0.75$: double to $m = 16$
5	$5 \bmod 16 = 5$	slot 5	no ($\frac{8}{16} = 0.5$)

Rehash to $m = 8$: $12 \to 4$, $43 \to 3$, $13 \to 5$, $88 \to 0$:

$$[88 \mid \cdot \mid \cdot \mid 43 \mid 12 \mid 13 \mid \cdot \mid \cdot] \tag{4}$$

Rehash to $m = 16$: $88 \to 8$, $43 \to 11$, $12 \to 12$, $13 \to 13$, $23 \to 7$, $26 \to 10$, $11 \to 11, 12, 13$ busy $\to 14$.

Final table ($m = 16$, $\cdot$ = empty):

slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
key	•	•	•	•	•	**5**	•	**23**	**88**	•	**26**	**43**	**12**	**13**	**11**	•

6.

(a) Some 3 of the 4 keys hash to the same slot and the fourth elsewhere:

$$P = \binom{4}{3} \cdot \left(\frac{1}{m}\right)^2 \cdot \left(1 - \frac{1}{m}\right) = \frac{4(m-1)}{m^3}. \tag{5}$$

(b) "At least 4" is the complement of "exactly 3", which requires k_2 to avoid k_1's slot $(1 - \frac{1}{m})$ and k_3 to avoid both $(1 - \frac{2}{m})$:

$$P(\text{total} \geq 4) = 1 - \left(1 - \frac{1}{m}\right)\left(1 - \frac{2}{m}\right) = \frac{3m - 2}{m^2}. \tag{6}$$

(c) Expected probes for an unsuccessful search: $\frac{1}{1-\alpha}$.

$$\frac{1}{1-\alpha} = \frac{1}{\frac{1}{\sqrt{n}}} = \sqrt{n} = \Theta\big(\sqrt{n}\big). \tag{7}$$

7.

(a) **Insertions.** Charge 3 per insertion: 1 for the insert, 2 stored as credit on the new element. Right after a doubling, the table of size m holds $\frac{m}{2}$ elements. The next doubling occurs only after at least $\frac{m}{2}$ more insertions, which bank $2 \cdot \frac{m}{2} = m$ credits — enough to pay the $\Theta(m)$ cost. Hence amortized cost $O(1)$.

Deletions. Charge a sufficiently large constant c per deletion: 1 for the delete, $c - 1$ banked. Right after a halving, the table of size m holds $\frac{m}{2}$ elements. The next halving occurs only after at least $\frac{m}{4}$ deletions, which bank $(c-1) \cdot \frac{m}{4} = \Theta(m)$ credits — enough to pay the $\Theta(m)$ halving cost for a suitable constant c. (If a doubling happens first, it is paid by the insertion credits.) Hence amortized cost $O(1)$. □

(b) All leaves of a 2-3 tree are at depth $\leq \log_2 n$, so any insertion or deletion costs at most $c_1 \log n$ actual work.

Charging scheme. Charge each insertion $C \log n$ ($C = 2c_1$), and deposit the surplus as **one credit on every node the insertion touches** ($\leq 2 \log n + O(1)$ nodes, $O(1)$ work per node). After every insertion every touched node carries a credit; nodes are only created/restructured by insertions, so **every node always carries at least one credit**.

Deletions cost 0. A deletion touches $\leq 2 \log n$ nodes ($O(1)$ work each); each touched node's own credit pays for that work, so the whole actual cost is paid from banked credits and the deletion is charged 0.

Balance. Each insertion deposits $\geq c_1 \log n$ credits; each deletion withdraws at most $c_1 \log n$ (its touched nodes' credits), and each node's credit is spent at most once before being re-armed by a later insertion that pays for it. With C large enough the credit balance never goes negative, so amortized $O(\log n)$ for insertion and 0 for deletion are valid. $\square$